Under the Hood: How Python Stores Your Data

Class 6 — In-Class Work-Along Sheet (Part 1 — CPython Memory Model)

Part 1 — List: The Pointer Array

Every Python list slot stores a **memory address**, not a value.

That single fact explains how lists hold mixed types — and why they over-allocate.

Q1 — Measuring a list's footprint

Run each line separately and record the output:

```
import sys
print(sys.getsizeof([]))
print(sys.getsizeof([1]))
print(sys.getsizeof([1, 2]))
print(sys.getsizeof([1, 2, 3]))
print(sys.getsizeof([1, 2, 3, 4]))
```

| Expression | Size (bytes) | |---|---| | `[]` | _____ | | `[1]` | _____ | | `[1, 2]` | _____ | | `[1, 2, 3]` | _____ | | `[1, 2, 3, 4]` | _____ |

(a) How many bytes does each additional element add? _____

(b) This tells you every list slot stores a pointer of size _____ bytes — regardless of whether it points to an `int`, a `str`, or anything else.

Q2 — Over-allocation: when does the list actually grow?

```
import sys

lst = []
for i in range(17):
    lst.append(i)
    print(f"len={len(lst):2d}  size={sys.getsizeof(lst)} bytes")
```

(a) At which lengths does the size **jump**? Mark them: _____

(b) Between jumps the size stays flat even though you added an element. What is CPython doing?

_____

(c) Why is this better than resizing on every single append?

_____

Q3 — Mixed types: the real reason

A list `[42, "hello", True, 3.14]` holds four completely different types.

(a) In Java or C++, an array must hold one type. Python lists don't. What is it about the pointer model that makes this possible?

_____

(b) ⟳ Back to Class 5 — **which other collection also stores only pointers/references** to objects (meaning the same object can appear in two different collections at once, and they see the same mutation)?

Part 2 — String: Compact Array + The Unicode Story

A string `"hello"` stores characters directly — no pointers. That's why it's so compact.

But which encoding? That depends on the widest character in the string.

Q4 — Characters are numbers (↻ from Class 5)

(a) Without running code: `ord('A')` = _____ · `ord('a')` = _____ · `ord('0')` = _____

(b) Now run:

```
print(ord('😀'))
print(hex(ord('😀')))
```

`ord('😀')` = _____ `hex(ord('😀'))` = _____

So 😀 is Unicode code point **U+**_____

Q5 — Hexadecimal: decoding U+1F600

Convert **1F600** from hex to decimal. Work through each digit:

```
1 × 16⁴  =  1 × _____  =  _____
F × 16³  = 15 × _____  =  _____
6 × 16²  =  6 × _____  =  _____
0 × 16¹  =              =      0
0 × 16⁰  =              =      0
              Total     =  _____
```

Confirm with `print(ord('😀'))` → _____ ✓

Bonus: `print(hex(128512))` → _____ (Python's notation for hex uses `0x` instead of `U+`)

Q6 — The bit-math of encoding sizes

Fill in the table from memory, then check with Python:

Bytes	Bits	2^bits = max values	Covers
1	8	_____	ASCII (0–127) + Latin-1 (0–255)
2	16	_____	most modern languages
4	32	_____	all of Unicode + emoji + future

(a) `ord('é')` = _____ · Is it > 255? _____ · So é needs _____ byte(s) per char

(b) `ord('α')` = _____ · Is it > 255? _____ · Is it > 65535? _____ · So α needs _____ byte(s)

(c) 😀 has code point 128512. Is it > 65535? _____ · So 😀 needs _____ byte(s)

(d) CPython's rule: scan the whole string → find the _____ code point → use that many bytes for **every** character in the string.

Q7 — sys.getsizeof: measuring string memory

Run:

```
import sys
print(sys.getsizeof("hello"))
print(sys.getsizeof("héllo"))
print(sys.getsizeof("αello"))
print(sys.getsizeof("😀ello"))
```

Record outputs: _____ · _____ · _____ · _____

Now fill in the formula for each string (each has 5 characters):

```
"hello"  →  ___ header  +  5 × ___ bytes/char  +  ___ null  =  ___ bytes
"héllo"  →  ___ header  +  5 × ___ bytes/char  +  ___ null  =  ___ bytes
"αello"  →  ___ header  +  5 × ___ bytes/char  +  ___ null  =  ___ bytes
"😀ello" →  ___ header  +  5 × ___ bytes/char  +  ___ null  =  ___ bytes
```

(a) The header jumps from 40 to 56 bytes between "hello" and "héllo", even though **both** use 1 byte per character. What extra 16 bytes does the non-ASCII header store?

_____

(b) Why does immutability make the fixed-width encoding possible? What would have to happen if you could write `s[0] = '😀'` on an ASCII string?

_____

Q8 — Inside the header: 40 bytes of overhead

Every Python object carries a fixed header. Name what each field does:

Field	Size	Purpose
`ob_refcnt`	8 bytes	_____ (used for garbage collection)
`ob_type`	8 bytes	_____ (enables `type(s)` and method lookup)
`length`	8 bytes	_____ (means `len(s)` is always _____)
`hash`	8 bytes	_____ (cached after first call → `hash(s)` is _____)
state / flags	4 bytes	stores encoding kind + interned flag
alignment	4 bytes	keeps memory on 8-byte boundary

A list `["h", "e", "l", "l", "o"]` stores 5 separate string objects (each ~50 bytes) plus 5 pointers. A string `"hello"` stores the same 5 characters in **46 bytes total**. Why is the string so much more compact?

_____

Q9 — The pipeline: from bits to screen

Complete the pipeline for `s = "α"`:

```
Your code writes:  s = "α"
          ↓
Code point:  _______  (the integer assigned to α)
          ↓
Assigned by:  ___________________________________ (which organisation?)
          ↓
CPython stores:  _______ bytes per character  (because α's code point > 255)
          ↓
To display — OS text rendering engine
(CoreText on Mac · DirectWrite on Windows · FreeType on Linux)
          ↓
________ file: maps code point 945 → vector curves for the shape α
          ↓
Font renderer: converts curves to ________ at the right size
          ↓
GPU draws to screen →  you see:  α
```

(a) Why does 😀 look **different** on iPhone vs Android, even though both devices use code point U+1F600?

_____

(b) If you open a document with a character whose code point exists in Unicode but no installed font covers it, what do you see?

_____

Q10 — Font demo

Your instructor will open `font_demo.html` in Chrome (or you can open it yourself — it's in the class_06 folder).

(a) What does the **Helvetica Neue** row show for the Egyptian hieroglyphs?

_____

(b) What does the **Noto Egyptian Hieroglyphs** row show?

_____

(c) Both rows have **identical code points**. What is the only thing that changed?

_____

(d) Complete: the _____ defines the number; the _____ defines the picture.

Part 3 — Set: The Hash Table

O(1) isn't magic — it's a trade-off: more memory for a faster lookup.

Q11 — hash() in Python

Run each line and record the output:

```
print(hash(42))
print(hash(42))           # run again
print(hash(3.14))
print(hash((1, 2, 3)))    # tuple: hashable
```

Now predict, then run:

```
print(hash([1, 2, 3]))    # list: predict crash or value?
```

(a) Is `hash(42)` the same on every call and in every Python session? _____

(b) Now run `hash("hello")` in **two separate terminal sessions**. Are they the same? _____ — Why does Python randomize string hashes between runs?

_____

(c) What error does `hash([1, 2, 3])` raise? _____ Why can't a list be hashed?

_____

(d) ⟳ From Class 5 Part 1 — you learned that set elements must be **hashable**. Now you know *why*: if an element's hash changed after it was added, _____

Q12 — Hash table mechanics

Given a hash table with 8 slots (0–7), trace the insertions for `{10, 25, 37}`:

```
hash(10) mod 8 = _______  →  stored in slot _______
hash(25) mod 8 = _______  →  stored in slot _______
hash(37) mod 8 = _______  →  stored in slot _______
```

To check `42 in {10, 25, 37}`: 1. Compute `hash(42) mod 8` = _____ 2. Check that slot — what is there? _____ 3. Result: _____ (True / False)

How many elements did Python have to examine? _____ — compare this to a list scan of 3 elements that doesn't find 42: how many comparisons? _____

Q13 — Memory trade-off: set vs list

```
import sys
n = 1000
lst = list(range(n))
s   = set(range(n))
print(f"list: {sys.getsizeof(lst):,} bytes")
print(f"set:  {sys.getsizeof(s):,} bytes")
```

(a) list: _____ bytes · set: _____ bytes · ratio ≈ _____×

(b) A hash table works best when it stays roughly **⅔ full**. What happens to lookup performance if the table is 100% full?

_____

(c) When a set fills past ⅔ capacity, CPython rebuilds it at double the size. This rebuild is O(n) — but why is the **average cost per insertion** still O(1)?

_____

Q14 — Synthesis: the full trade-off table (◌ books closed)

Fill this in from memory:

	`list`	`set`	`str`
`x in container`	O(____)	O(____)	O(____)
Index by position `[i]`	O(1)	✗	O(1)
Memory layout	_____ array	sparse _____	compact _____
Ordered?	✓	✗	✓
Mutable?	✓	✓	✗
Elements must be hashable?	✗	✓	N/A

Q15 — ◌ Retrieval (books closed, no hints)

(a) A Python list `[42, "hi", True]` uses _____ bytes per slot regardless of the type stored there. Explain why in one sentence: _____

(b) `sys.getsizeof("hello")` = 46. The string has 5 characters × 1 byte = 5 bytes of data. What is in the remaining **41 bytes**? Name at least three fields.

_____

(c) Two strings: `"hello"` and `"α"`. Same length (5 and 1), completely different memory layouts. What determines how many bytes per character CPython uses?

_____

(d) U+1F600 — without running code, compute `ord('😀')` from the hex digits. Show working:

_____

(e) ◌ From Class 5 Part 6 — you measured a ~100,000× speedup for `in` on a set vs a list. Explain it now precisely: _____

(f) You want to store 1 million product SKUs and check membership thousands of times per second. List or set? _____ — and what is the cost you pay in exchange?

_____

Part 2 — Input, Output, and Beyond the Console

> The console is just the default. The same byte-stream mechanism reaches files and networks.

Q16 — `input()` always returns `str`

(a) Predict the type, then run:

```
x = input("Enter a number: ")
print(type(x))
```

Type of `x`: _____

(b) Now predict whether this crashes, then run:

```
x = input("Enter a number: ")
result = x + 10
print(result)
```

Prediction: crash / runs fine? _____ · Error type if crash: _____

↻ Which bug from Class 5's debugging lab does this mirror exactly? _____

(c) Fix it with a conversion:

```
x = int(input("Enter a number: "))
result = x + 10
print(result)
```

What happens if the user types `"hello"` instead of a number? _____ ↻ This error type appeared in Class 2. What is it called? _____

(d) Write the one-liner that reads a **float** from the user and stores it in `price`:

```
price = _____________________________
```

Q17 — Build something: birthday calculator

Write a program that: 1. Asks for the user's first name 2. Asks for their birth year (as an integer) 3. Prints: `Hello <name>! You were born in <year>. You are approximately <age> years old in 2026.`

```
# your code here
```

Run it with your own name and year. Does it work? _____

↻ From Class 5 Bug 2: `"Total SKUs: " + total_skus` crashed with a TypeError. Explain how `int(input(...))` prevents the same class of bug when reading numbers from users:

_____

Q18 — `print()` formally: `sep` and `end`

(a) Predict, then run:

```
print("Mon", "Tue", "Wed")
```

Output: _____

Now with `sep`:

```
print("Mon", "Tue", "Wed", sep=" | ")
```

Output: _____

And:

```
print("Mon", "Tue", "Wed", sep="\n")
```

Output (describe what you see): _____

(b) Predict, then run:

```
print("Loading", end="")
print("...")
```

Output: _____

vs the default:

```
print("Loading")
print("...")
```

Output: _____

(c) Use **one** `print()` call with `sep=` to produce this exact output:

`2026/06/26`

```
print(_______, _______, _______, sep=_______)
```

(d) Use **three** separate `print()` calls with `end=` to produce this on one line:

`1 2 3`

```
print(1, end=___)
print(2, end=___)
print(3)
```

Q19 — f-strings: format specs

(a) Float precision — run and record:

```
pi = 3.14159265358979
print(f"raw:      {pi}")
print(f"2 places: {pi:.2f}")
print(f"5 places: {pi:.5f}")
```

Outputs: _____ · _____ · _____

(b) Thousands separator — run and record:

```
population = 1_400_000_000
print(f"raw:    {population}")
print(f"commas: {population:,}")
```

Outputs: _____ · _____

(c) Width and alignment — run:

```
for name, price in [("Apple", 45.5), ("Banana", 12.0), ("Dragonfruit", 310.75)]:
    print(f"{name:<15} Rs.{price:>8.2f}")
```

What does `:<15` do? _____

What does `:>8.2f` do? _____

(d) Zero padding — predict each output before running:

```
for n in [1, 12, 123]:
    print(f"Order #{n:04d}")
```

```
Order #____
Order #____
Order #____
```

(e) Put it together — fill in the one blank to complete this receipt printer:

```
items = [("Laptop", 1, 75000), ("USB Cable", 3, 499.50)]
print(f"{'Item':<15} {'Qty':>4}   {'Price':>12}")
print("-" * 35)
for name, qty, price in items:
    print(f"{name:<15} {qty:>4}   Rs. {price:________}")
```

Expected output:

```
Item             Qty          Price
-----------------------------------
Laptop             1   Rs. 75,000.00
USB Cable          3   Rs.    499.50
```

The blank: `________` → fill in the format spec for price.

Q20 — The big picture: beyond the console

(a) Fill in the blanks:

`input()` reads from _____ (standard _____). `print()` writes to _____ (standard _____). Together, these form the **console** interface.

(b) `print()` has a `file=` argument. What do you think this does?

```
with open("output.txt", "w") as f:
    print("Hello, file!", file=f)
```

Your prediction (no need to run): _____

(c) Complete the analogy:

```
Console:  input() reads from keyboard  → program → print() writes to screen
File:     _____________ reads from file → program → print(..., file=f) writes to _______
Network:  socket.recv() reads from ___ → program → socket.send() writes to _______
```

(d) A stream is just: _____

The console is one possible endpoint. What is the same across console, file, and network?

_____

Part 3 — if / elif / else

Conditions you can evaluate. Branches you can now act on.

Q21 — Python vs Java / C++

The same logic in two languages:

```
// Java
if (x > 0) {
    System.out.println("Positive");
    System.out.println("Take action");
}
// indentation here is purely cosmetic — Java ignores it
```

Fill in the equivalent Python:

```
if _________:
    print("Positive")
    ____________________________
# No _______ needed — _______________ defines the block
```

(a) In Java, a block is defined by _____. In Python, by _____.

(b) What happens in Python if you remove the 4-space indent from `print("Positive")`? _____

(c) Can a Python if-block contain zero lines of indented code? _____ (Hint: look up `pass`)

Q22 — Indentation rules

For each snippet, predict: **Runs correctly / IndentationError / SyntaxError**

(a)

```
x = 10
if x > 5:
print("big")
```

Prediction: _____

(b)

```
x = 10
if x > 5:
    print("big")
    print("number")
```

Prediction: _____

(c)

```
x = 10
if x > 5:
    print("positive")
print("always runs")     # ← same level as if
else:
    print("non-positive")
```

Prediction: _____ · What rule does this break? _____

(d)

```
x = 10
if x > 5:
    print("big")
  print("also big")      # ← 2 spaces, not 4
```

Prediction: _____

Q23 — The anti-pattern: why `if` / `if not` is fragile

Trace step by step with `stock = 8`:

```
stock = 8

if stock < 10:
    print("Low stock alert!")
stock = 15                      # modified between the two checks
if not (stock < 10):
    print("Stock is fine.")
```

(a) Which `print` lines actually run? _____

(b) Did we intend **both** to run? _____ Is this a bug? _____

(c) Rewrite using `if-else` (nothing between them):

```
stock = 8
if stock < 10:
    _____________
else:
    _____________
```

(d) With `if-else`, can both branches ever run? _____ This is why `if-else` is the right tool when you want _____.

Q24 — Match the pattern to its type

Patterns:

```
P1:  if age >= 18:              P2:  if score >= 90:
         print("Adult")                 print("A")
     else:                          elif score >= 80:
         print("Minor")                 print("B")
                                    else:
                                        print("C")

P3:  if a > 0:                  P4:  if x > 0:
         if b > 0:                      print("positive")
             print("Both +")        if x % 2 == 0:
         else:                          print("even")
             print("a only")       if x > 100:
     else:                              print("large")
         print("a not +")

P5:  if premium:
         print("Discount applied")
     if loyalty_years > 5:
         print("Loyalty bonus")
```

Types: A — simple if · B — if-else · C — elif ladder · D — multiple independent if · E — nested if

P1 → ____ · P2 → ____ · P3 → ____ · P4 → ____ · P5 → ____

Practice Problems

Read from stdin · Write to stdout · Difficulty in [brackets]

Q25 · Category A · [Easy] — Vote eligibility

Read an integer age. If age ≥ 18, print `You can vote.` Print nothing otherwise.

```
Input: 20   →   Output: You can vote.
Input: 15   →   Output: (nothing)
```

```
# your solution
```

Q26 · Category A · [Medium] — Loyal customer discount

Read a purchase amount (integer, rupees). If > Rs.1000, apply 10% discount and print `Discounted price: Rs.X.XX`. Print nothing otherwise.

```
Input: 1500   →   Output: Discounted price: Rs.1350.00
Input: 800    →   Output: (nothing)
```

```
# your solution
```

Q27 · Category A · [Hard] — Merit scholarship

A score is out of 150. Read the score (integer). Compute percentage. If ≥ 90%, print `Merit scholarship awarded. Percentage: X.XX%`. Print nothing otherwise.

```
Input: 135   →   Output: Merit scholarship awarded. Percentage: 90.00%
Input: 120   →   Output: (nothing)
```

```
# your solution
```

Q28 · Category A · [Hard] — Distinction checker

Read three quiz scores (integers 0–100), one per line. If the average is **strictly > 80 AND all three scores ≥ 60**, print `Distinction awarded.` Print nothing otherwise.

```
Input: 85 / 90 / 88   →   Output: Distinction awarded.
Input: 95 / 55 / 90   →   Output: (nothing)   ← one score < 60
Input: 75 / 78 / 80   →   Output: (nothing)   ← average not > 80
```

```
# your solution
```

Q29 · Category B · [Easy] ★ First LMS problem — Even or odd

Read an integer. Print `Even` or `Odd`.

```
Input: 4   →   Even
Input: 7   →   Odd
Input: 0   →   Even
```

```
# your solution
```

Q30 · Category B · [Medium] — Electricity bill (2-slab)

Read units consumed (integer). Compute and print the bill:

- ≤ 100 units: Rs. 1.50 per unit
- > 100 units: Rs. 2.50 per unit (for **all** units)

Print: `Bill: Rs.X.XX`

```
Input: 80    →   Bill: Rs.120.00
Input: 150   →   Bill: Rs.375.00
```

```
# your solution
```

Q31 · Category B · [Hard] — Leap year (full rule)

A year is a leap year if: divisible by 4, AND not divisible by 100, UNLESS also divisible by 400.

```
Input: 2024   →   Leap year
Input: 1900   →   Not a leap year   ← div by 100, not 400
Input: 2000   →   Leap year         ← div by 400
Input: 2023   →   Not a leap year
```

```
# your solution
```

Q32 · Category B · [Hard] — Taxi fare

Read a distance in km (float). Compute fare:

- 0 km → `No trip.`
- ≤ 2 km → flat Rs. 50
- > 2 km → Rs. 50 + Rs. 15 × (km − 2)

Print: `No trip.` or `Fare: Rs.X.XX`

```
Input: 0     →   No trip.
Input: 1.5   →   Fare: Rs.50.00
Input: 5.0   →   Fare: Rs.95.00
```

```
# your solution
```

Q33 · Category C · [Easy] — Number tags

Read an integer. Print each applicable tag (0 to 3 lines):

- If positive → `Positive`
- If even → `Even`
- If > 100 → `Large`

```
Input: 150   →   Positive / Even / Large   (3 lines)
Input: -4    →   Even                      (1 line)
Input: 7     →   Positive                  (1 line)
Input: -3    →   (nothing)
```

```
# your solution
```

Q34 · Category C · [Medium] — Student badges

Read a score and attendance % (integers), one per line. Print each applicable badge:

- score > 90 → `Star student`
- attendance > 90 → `Perfect attendance`

```
Input: 95 / 95   →   Star student + Perfect attendance
Input: 95 / 80   →   Star student
Input: 70 / 70   →   (nothing)
```

```
# your solution
```

Q35 · Category C · [Hard] — Product tags

Read four values (one per line): price (Rs.), stock quantity, days since launch, total units sold. Print each applicable tag:

- `On sale` — price < 500
- `Low stock` — quantity < 10
- `New arrival` — days since launch < 30
- `Bestseller` — units sold > 1000

```
Input: 399 / 5 / 10 / 1500   →   On sale + Low stock + Bestseller
Input: 999 / 100 / 100 / 50  →   (nothing)
```

```
# your solution
```

Q36 · Category C · [Hard] — Quiz bonuses

Read three quiz scores (integers 0–100), one per line. Print each applicable bonus:

- `Perfect score bonus` — any score == 100
- `Consistency bonus` — all three > 80
- `High achiever bonus` — sum > 250

```
Input: 100 / 90 / 85   →   all three bonuses
Input: 100 / 70 / 60   →   Perfect score bonus only
Input: 60 / 70 / 80    →   (nothing)
```

```
# your solution
```

Q37 · Category D · [Easy] — Grade calculator

Read a score (0–100). Print the grade: 90+ → A · 80+ → B · 70+ → C · 60+ → D · else → F

```
Input: 95   →   A
Input: 83   →   B
Input: 45   →   F
```

```
# your solution
```

Q38 · Category D · [Medium] — Electricity bill (4-slab)

Each slab rate applies only to units in that range:

- 1–100 → Rs. 3/unit · 101–200 → Rs. 5/unit · 201–300 → Rs. 7/unit · 301+ → Rs. 10/unit

Print: `Bill: Rs.X.XX`

```
Input: 80    →   Bill: Rs.240.00
Input: 150   →   Bill: Rs.550.00
Input: 250   →   Bill: Rs.1,150.00
Input: 350   →   Bill: Rs.2,000.00
```

```
# your solution
```

Q39 · Category D · [Hard] — Maximum of three (no `max()`)

Read three integers, one per line. Print the largest. Do **not** use `max()`.

```
Input: 4 / 9 / 2     →   9
Input: -1 / -5 / -3  →   -1
Input: 7 / 7 / 3     →   7
```

```
# your solution
```

Q40 · Category D · [Hard] — Income tax calculator

Progressive tax (each slab rate applies to the amount in that slab only):

| Slab | Rate | |------|------| | First Rs. 2,50,000 | 0% | | Rs. 2,50,001 – Rs. 5,00,000 | 5% | | Rs. 5,00,001 – Rs. 10,00,000 | 20% | | Above Rs. 10,00,000 | 30% |

Read annual income (integer). Print: `Tax payable: Rs.X.XX`

```
Input: 200000    →   Tax payable: Rs.0.00
Input: 400000    →   Tax payable: Rs.7,500.00
Input: 700000    →   Tax payable: Rs.52,500.00
Input: 1500000   →   Tax payable: Rs.2,62,500.00
```

```
# your solution
```

Q41 · Category E · [Easy] — Loan eligibility

Read age and monthly income (integers), one per line.

- age < 18 → `Not eligible: age requirement not met.`
- age ≥ 18 and income < 30000 → `Not eligible: income too low.`
- age ≥ 18 and income ≥ 30000 → `Loan approved.`

```
Input: 25 / 35000   →   Loan approved.
Input: 25 / 20000   →   Not eligible: income too low.
Input: 16 / 50000   →   Not eligible: age requirement not met.
```

```
# your solution
```

Q42 · Category E · [Medium] — Coordinate quadrant

Read x and y (integers), one per line. Print: `Origin` · `X-axis` · `Y-axis` · `Quadrant I/II/III/IV`

```
Input: 3 / 4    →   Quadrant I
Input: -2 / 5   →   Quadrant II
Input: 0 / 0    →   Origin
Input: 5 / 0    →   X-axis
```

```
# your solution
```

Q43 · Category E · [Hard] — Max and min of four (no `max()` / `min()`)

Read four integers, one per line. Print max then min on separate lines. Do **not** use `max()` or `min()`.

```
Input: 4 / 9 / 2 / 7      →   Max: 9 / Min: 2
Input: -1 / -5 / -3 / -2  →   Max: -1 / Min: -5
```

```
# your solution
```

Q44 · Category E · [Hard] — Rock-Paper-Scissors

Read two players' choices (`rock` / `paper` / `scissors`), one per line. Use only nested `if` (no `and` in conditions). Print `Player 1 wins` , `Player 2 wins` , or `Draw` .

Rules: rock beats scissors · scissors beats paper · paper beats rock

```
Input: rock / scissors   →   Player 1 wins
Input: paper / paper     →   Draw
Input: scissors / rock   →   Player 2 wins
```

```
# your solution
```